Deadlock Detector API 문서

개요

Deadlock Detector는 운영체제의 데드락 감지 및 회복 알고리즘을 AI/LLM 다중 에이전트 시스템에 적용한 시스템입니다. Wait-For Graph 기반 사이클 탐지와 다양한 희생자 선택 전략을 제공합니다.

기본 **URL**: `http://localhost:3003/api`

---

1. 시스템 상태 (System Status)

1.1 헬스 체크

```
GET /api/health
```

응답 예시:

```
{
  "status": "ok",
  "timestamp": "2025-01-25T10:30:00.000Z",
  "services": {
    "mongodb": "connected",
    "redis": "connected"
  }
}
```

1.2 그래프 상태 조회

현재 Wait-For Graph 상태를 확인합니다.

```
GET /api/graph
```

응답 예시:

```
{
  "agents": [
    { "id": "agent-1", "name": "Agent 1", "priority": 5 },
    { "id": "agent-2", "name": "Agent 2", "priority": 3 }
  ],
  "resources": [
    { "id": "res-1", "name": "Resource 1", "instances": 1 }
  ],
  "edges": [
    { "from": "agent-1", "to": "res-1", "type": "waiting" }
  ],
  "cycles": []
}
```

---

2. 에이전트 관리 (Agent Management)

2.1 에이전트 생성

새로운 에이전트를 생성합니다.

```
POST /api/agents
Content-Type: application/json
```

요청 본문:

```
{
  "name": "Agent-1",
  "priority": 5,
  "metadata": {
    "type": "worker",
    "createdBy": "admin"
  }
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |---|---|---|---| | name | string | Y | 에이전트 이름 | | priority | number | N | 우선순위 (1-10, 기본값: 5) | | metadata | object | N | 추가 메타데이터 |

응답 예시:

```
{
  "success": true,
  "agent": {
    "id": "agent-1",
    "name": "Agent-1",
    "priority": 5,
    "status": "active",
    "createdAt": "2025-01-25T10:30:00.000Z"
  }
}
```

2.2 에이전트 목록 조회

```
GET /api/agents
```

2.3 에이전트 삭제

```
DELETE /api/agents/:agentId
```

---

3. 자원 관리 (Resource Management)

3.1 자원 생성

새로운 자원을 생성합니다.

```
POST /api/resources
Content-Type: application/json
```

요청 본문:

```
{
  "name": "Resource-1",
  "type": "computational",
  "instances": 1,
  "metadata": {
    "description": "GPU resource"
  }
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |---|---|---|---| | name | string | Y | 자원 이름 | | type | string | N | 자원 유형 (computational, storage, network) | | instances | number | N | 인스턴스 수 (기본값: 1) | | metadata | object | N | 추가 메타데이터 |

응답 예시:

```
{
  "success": true,
```

```
  "resource": {
    "id": "res-1",
    "name": "Resource-1",
    "type": "computational",
    "instances": 1,
    "available": 1,
    "createdAt": "2025-01-25T10:30:00.000Z"
  }
}
```

3.2 자원 요청

에이전트가 자원을 요청합니다.

```
POST /api/resources/request
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-1",
  "resourceId": "res-1"
}
```

응답 예시:

```
{
  "success": true,
  "message": "Resource allocated successfully",
  "allocation": {
    "agentId": "agent-1",
    "resourceId": "res-1",
    "allocatedAt": "2025-01-25T10:30:00.000Z"
  }
}
```

3.3 자원 해제

에이전트가 자원을 해제합니다.

```
POST /api/resources/release
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-1",
  "resourceId": "res-1"
}
```

응답 예시:

```
{
  "success": true,
  "message": "Resource released successfully"
}
```

---

4. 데드락 감지 및 회복 (Deadlock Detection & Recovery)

4.1 데드락 감지

Wait-For Graph에서 사이클(데드락)을 감지합니다.

```
POST /api/deadlock/detect
```

응답 예시:

```
{
  "hasDeadlock": true,
  "cycles": [
    {
      "cycleId": "cycle-1",
      "agents": ["agent-1", "agent-2", "agent-3"],
      "resources": ["res-1", "res-2", "res-3"],
      "detectedAt": "2025-01-25T10:30:00.000Z"
    }
  ],
  "statistics": {
    "totalAgents": 3,
    "totalResources": 3,
    "totalEdges": 3,
    "cyclesFound": 1
  }
}
```

4.2 희생자 선택

데드락 해결을 위해 종료할 에이전트(희생자)를 선택합니다.

```
POST /api/deadlock/victim
Content-Type: application/json
```

요청 본문:

```
{
  "cycleId": "cycle-1",
  "strategy": "lowest_priority"
}
```

지원하는 전략: - `lowest_priority`: 가장 낮은 우선순위 에이전트 선택 - `youngest`: 가장 최근에 생성된 에이전트 선택 - `most_resources`: 가장 많은 자원을 보유한 에이전트 선택 - `least_dependencies`: 가장 적은 의존성을 가진 에이전트 선택

응답 예시:

```
{
  "success": true,
  "victim": {
    "agentId": "agent-3",
    "name": "Agent 3",
    "priority": 1,
    "resources": ["res-3"],
    "selectedAt": "2025-01-25T10:30:00.000Z"
  },
  "strategy": "lowest_priority",
  "reason": "Agent has lowest priority (1) in the cycle"
}
```

---

5. 체크포인트 및 롤백 (Checkpoint & Rollback)

5.1 체크포인트 생성

에이전트의 현재 상태를 저장합니다.

```
POST /api/recovery/checkpoint/:agentId
```

응답 예시:

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "ckpt-abc123",
    "agentId": "agent-1",
    "timestamp": "2025-01-25T10:30:00.000Z",
    "state": {
      "heldResources": ["res-1"],
      "waitingFor": "res-2",
      "status": "waiting"
    }
  }
}
```

5.2 롤백 실행

저장된 체크포인트로 에이전트 상태를 복원합니다.

```
POST /api/recovery/rollback/:agentId
Content-Type: application/json
```

요청 본문:

```
{
  "checkpointId": "ckpt-abc123",
  "releaseResources": true
}
```

응답 예시:

```
{
  "success": true,
  "rollback": {
    "agentId": "agent-1",
    "restoredState": {
      "heldResources": [],
      "waitingFor": null,
      "status": "active"
    },
    "rollbackTime": 150
  }
}
```

---

6. 은행원 알고리즘 (Banker's Algorithm)

6.1 안전 상태 조회

현재 시스템이 안전한지 확인합니다.

```
GET /api/bankers
```

응답 예시:

```
{
  "isSafe": true,
  "availableResources": {
    "res-1": 2,
```

```
    "res-2": 1,
    "res-3": 3
  },
  "safeSequence": ["agent-1", "agent-2", "agent-3"],
  "analysis": {
    "totalAgents": 3,
    "totalResources": 3,
    "canFinishAll": true
  }
}
```

6.2 자원 할당 요청 (안전성 검사)

안전성을 보장하는 방식으로 자원을 요청합니다.

```
POST /api/bankers/request
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-1",
  "requests": {
    "res-1": 1,
    "res-2": 2
  }
}
```

응답 예시:

```
{
  "success": true,
  "isSafe": true,
  "allocated": {
    "res-1": 1,
    "res-2": 2
  },
  "message": "Resource allocation is safe"
}
```

---

7. WebSocket 이벤트 (Real-time Updates)

7.1 연결

```
const io = require('socket.io-client');
const socket = io('http://localhost:3003');
```

7.2 그래프 업데이트 구독

```
socket.emit('subscribe:graph');
```

7.3 데드락 알림 구독

```
socket.emit('subscribe:deadlock');
```

7.4 수신 이벤트

그래프 업데이트:

```
socket.on('graph-updated', (data) => {
  console.log('Graph updated:', data);
});
```

데드락 감지:

```
socket.on('deadlock-detected', (data) => {
  console.log('Deadlock detected:', data);
  // {
  //   cycleId: "cycle-1",
  //   agents: ["agent-1", "agent-2"],
  //   timestamp: "2025-01-25T10:30:00.000Z"
  // }
});
```

---

8. 에러 응답 (Error Responses)

8.1 에러 형식

```
{
  "success": false,
  "error": {
    "code": "AGENT_NOT_FOUND",
    "message": "Agent not found",
    "details": {
      "agentId": "agent-999"
    }
  }
}
```

8.2 에러 코드

코드	설명
VALIDATION_ERROR	요청 파라미터 검증 실패
AGENT_NOT_FOUND	에이전트를 찾을 수 없음
RESOURCE_NOT_FOUND	자원을 찾을 수 없음
RESOURCE_ALREADY_ALLOCATED	자원이 이미 할당됨
DEADLOCK_DETECTED	데드락이 감지됨
UNSAFE_STATE	안전하지 않은 상태

---

9. 사용 예제 (Usage Examples)

9.1 Node.js 클라이언트

```
const axios = require('axios');

class DeadlockDetectorClient {
  constructor(baseURL = 'http://localhost:3003/api') {
    this.client = axios.create({ baseURL });
  }

  // 에이전트 생성
  async createAgent(name, priority = 5) {
    const response = await this.client.post('/agents', { name, priority });
    return response.data.agent;
  }

  // 자원 요청
```

```
  async requestResource(agentId, resourceId) {
    const response = await this.client.post('/resources/request', {
      agentId,
      resourceId
    });
    return response.data;
  }

  // 데드락 감지
  async detectDeadlock() {
    const response = await this.client.post('/deadlock/detect');
    return response.data;
  }

  // 희생자 선택
  async selectVictim(cycleId, strategy = 'lowest_priority') {
    const response = await this.client.post('/deadlock/victim', {
      cycleId,
      strategy
    });
    return response.data.victim;
  }
}

// 사용 예시
const client = new DeadlockDetectorClient();

async function main() {
  try {
    // 에이전트 생성
    const agent1 = await client.createAgent('Agent 1', 5);
    const agent2 = await client.createAgent('Agent 2', 3);

    // 자원 생성 및 요청
    await client.createResource('Resource 1', 'computational');
    await client.requestResource(agent1.id, 'res-1');

    // 데드락 감지
    const deadlock = await client.detectDeadlock();
    if (deadlock.hasDeadlock) {
      console.log('Deadlock detected:', deadlock.cycles);

      // 희생자 선택
      const victim = await client.selectVictim(
        deadlock.cycles[0].cycleId,
        'lowest_priority'
      );
      console.log('Victim:', victim);
    }

  } catch (error) {
    console.error('Error:', error.response?.data || error.message);
  }
}

main();
```

10. OS 개념 매핑 (OS Concepts Mapping)

OS 개념	구현	설명
Wait-For Graph	WaitForGraph	에이전트와 자원 간의 대기 관계를 그래프로 표현
Cycle Detection	DFS (Depth-First Search)	그래프에서 사이클 탐지
Victim Selection	다양한 전략	데드락 해결을 위한 희생자 선택
Banker's Algorithm	Safety Checker	데드락 회피를 위한 안전 상태 검사
Checkpointing	상태 저장 및 복구	롤백을 위한 에이전트 상태 저장

---

11. 변경 로그 (Changelog)

v1.0.0 (2025-01-25)

- 초기 릴리스
- Wait-For Graph 기반 데드락 감지
- 4가지 희생자 선택 전략
- 은행원 알고리즘 구현
- 체크포인트 및 롤백 지원
- WebSocket 실시간 알림

문서 버전: 1.0.0 최종 업데이트: 2025-01-25 연락처: GitHub Issues